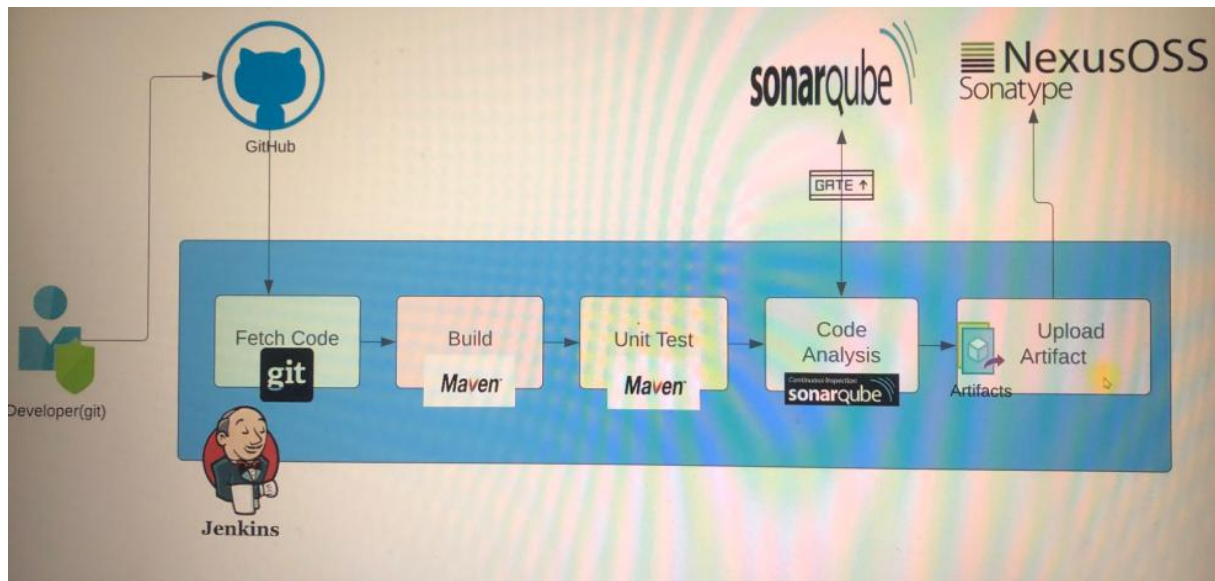


This documentation is done to describe all the process for building this pipeline using Jenkins



First step : install Jenkins

Firstly, launch an EC2 instance to run Jenkins

Name : jenkins-server

OS : Ubuntu

Amazon Machine Image : Ubuntu 24.04

Instance type : t2.small (we need at least 2Gio of memory)

Key pair : jenkins-key.pem

Security group : name(jenkins -sg), description(jenkins -sg)

Allow ssh inbound rule from MyIP

Allow Custom TCP with port 8080 from MyIP (To use it in the browser)

Launch the instance

Conect to the instance with ssh

Secondly, to install Jenkins, go the doc of Jenkins and get the command to install Jenkins on a ubuntu server : click

here(<https://www.jenkins.io/doc/book/installing/linux/>)

This is step of installing jenkins on ubuntu

```
sudo apt update
```

```
sudo apt install openjdk-21-jdk -y
```

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \  
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \  
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \  
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install jenkins -y
```

```
ubuntu@ip-172-31-81-198: ~  
elidj@Sougoumay MINGW64 ~/OneDrive/Bureau/DevOps/aws/project/vprofile-project (jenkins_CI)  
$ ssh -i "jenkins-key.pem" ubuntu@ec2-44-202-61-231.compute-1.amazonaws.com  
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:       https://ubuntu.com/pro  
  
System information as of Sun Jun 29 11:37:05 UTC 2025  
  
System load:  0.65          Processes:           106  
Usage of /:   25.8% of 6.71GB Users logged in:       0  
Memory usage: 11%          IPv4 address for enx0: 172.31.81.198  
Swap usage:   0%  
  
Expanded Security Maintenance for Applications is not enabled.  
  
0 updates can be applied immediately.  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The list of available updates is more than a week old.  
To check for new updates run: sudo apt update  
  
Last login: Sun Jun 29 11:36:21 2025 from 176.177.55.131  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@ip-172-31-81-198:~$ |
```

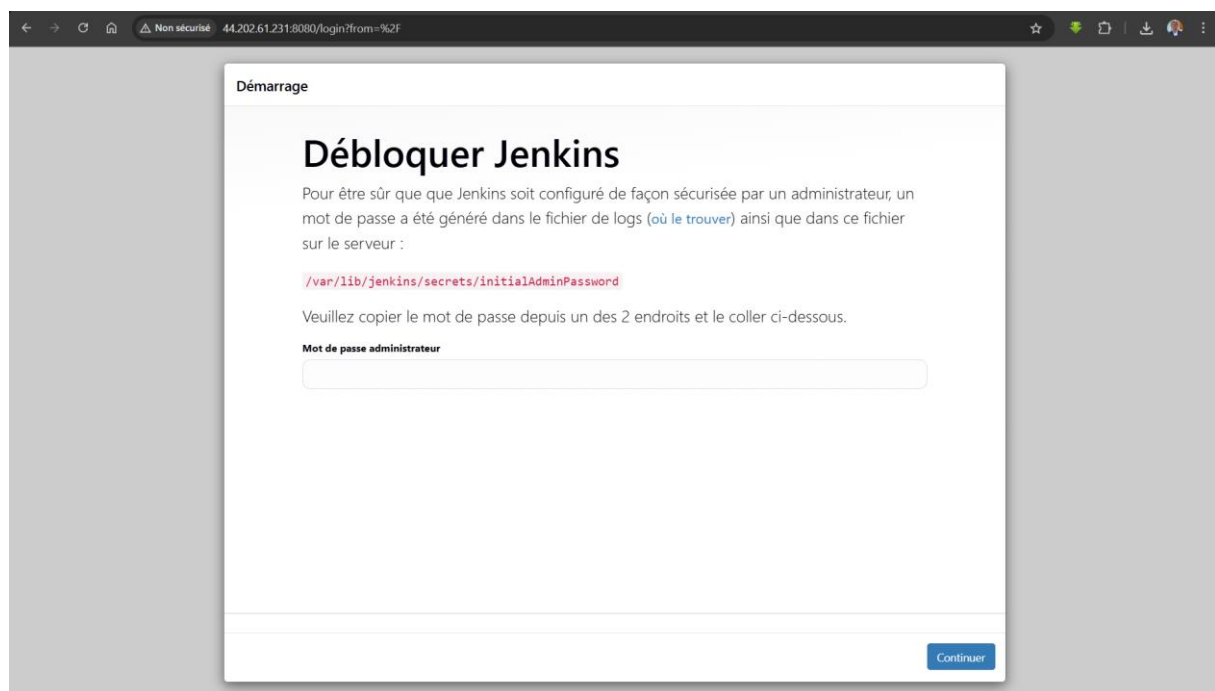
Switch as root user : `sudo -i`

Execute the commands below to install Jenkins on the server

Check Jenkins status to be sure it's running

```
ubuntu@ip-172-31-81-198: ~  
ubuntu@ip-172-31-81-198:~$ systemctl status jenkins  
● jenkins.service - Jenkins Continuous Integration Server  
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2025-06-29 11:43:46 UTC; 3min 32s ago  
 Main PID: 4115 (java)  
    Tasks: 37 (limit: 2332)  
  Memory: 365.3M (peak: 384.2M)  
     CPU: 19.041s  
    CGroup: /system.slice/jenkins.service  
            └─4115 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=>  
  
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: ec4a6d1c7a37470b9e76edf515a6901e  
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: This may also be found at: /var/lib/jenkins/secrets/in>  
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: >  
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: >  
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: >  
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.320+0000 [id=31] INFO >  
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.348+0000 [id=24] INFO >  
Jun 29 11:43:46 ip-172-31-81-198 systemd[1]: Started jenkins.service - Jenkins Continuous Integration >  
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.786+0000 [id=48] INFO >  
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.790+0000 [id=48] INFO >  
ubuntu@ip-172-31-81-198:~$ ls /var/lib/jenkins/  
config.xml      jobs             secret.key       updates  
hudson.model.UpdateCenter.xml  nodeMonitors.xml secret.key.not-so-secret userContent  
jenkins.telemetry.Correlator.xml plugins          secrets          users  
ubuntu@ip-172-31-81-198:~$
```

Check to connect to the server on the browser with port 8080. The next image is the first step



To have the password of the administrator, execute the next command

```
cat /var/lib/jenkins/secrets/initialAdminPassword :  
ec4a6d1c7a37470b9e76edf515a6901e
```

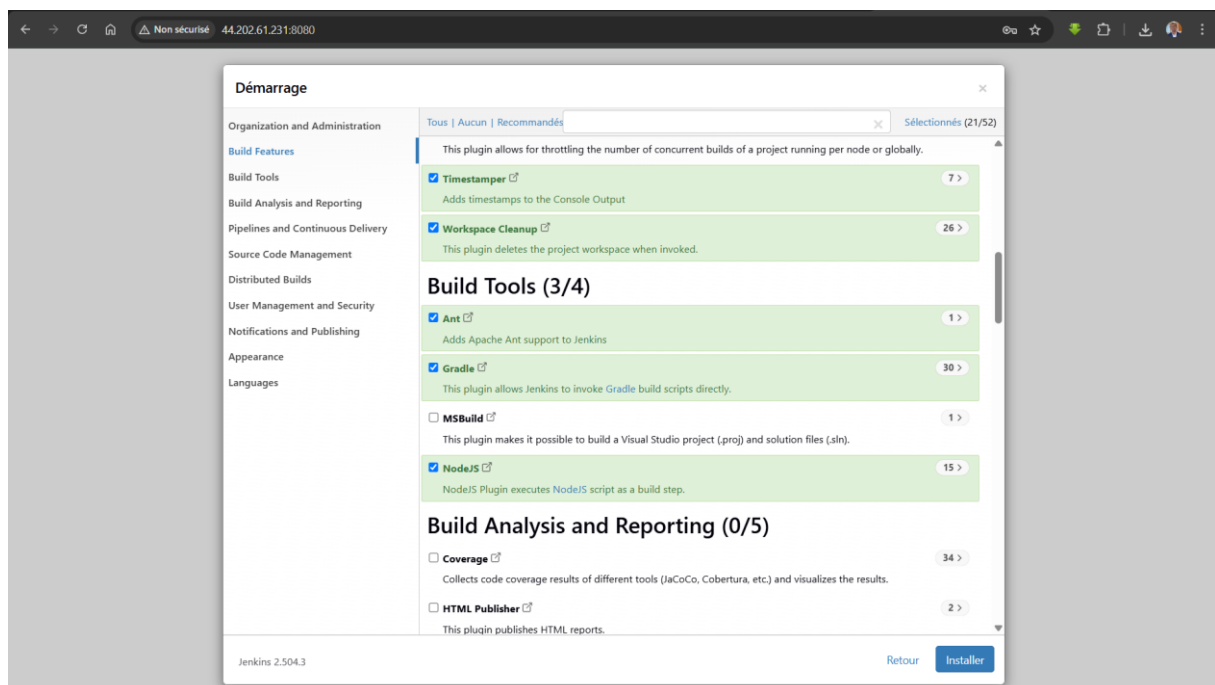
```
root@ip-172-31-81-198: ~  
ubuntu@ip-172-31-81-198:~$ cat /var/lib/jenkins/secrets/initialAdminPassword  
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied  
ubuntu@ip-172-31-81-198:~$ cat /var/lib/jenkins/secrets/initialAdminPassword  
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied  
ubuntu@ip-172-31-81-198:~$ sudo -i  
root@ip-172-31-81-198:~# cat /var/lib/jenkins/secrets/initialAdminPassword  
ec4a6d1c7a37470b9e76edf515a6901e  
root@ip-172-31-81-198:~#
```

The next step is this



Click to select plugin install

Select nodejs and click install



Installation en cours...

Installation en cours...

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	⌚ Credentials Binding	** Ionicons API
⌚ Timestampers	⌚ Workspace Cleanup	⌚ Ant	⌚ Gradle	Folders
⌚ Pipeline	⌚ GitHub Branch Source	⌚ Pipeline: GitHub Groovy Libraries	⌚ Pipeline Graph View	OWASP Markup Formatter
⌚ Git	⌚ SSH Build Agents	⌚ Matrix Authorization Strategy	○ PAM Authentication	** ASM API
⌚ LDAP	⌚ Email Extension	⌚ Mailer	○ Dark Theme	** JSON Path API
⌚ NodeJS				** Structs
				** Pipeline: Step API
				** Token Macro
				Build Timeout
				**ouncycastle API

Create the first admin user

Démarrage

Créer le 1er utilisateur Administrateur

Nom d'utilisateur

admin

Mot de passe

.....

Confirmation du mot de passe

.....

Nom complet

Admin

Adresse courriel

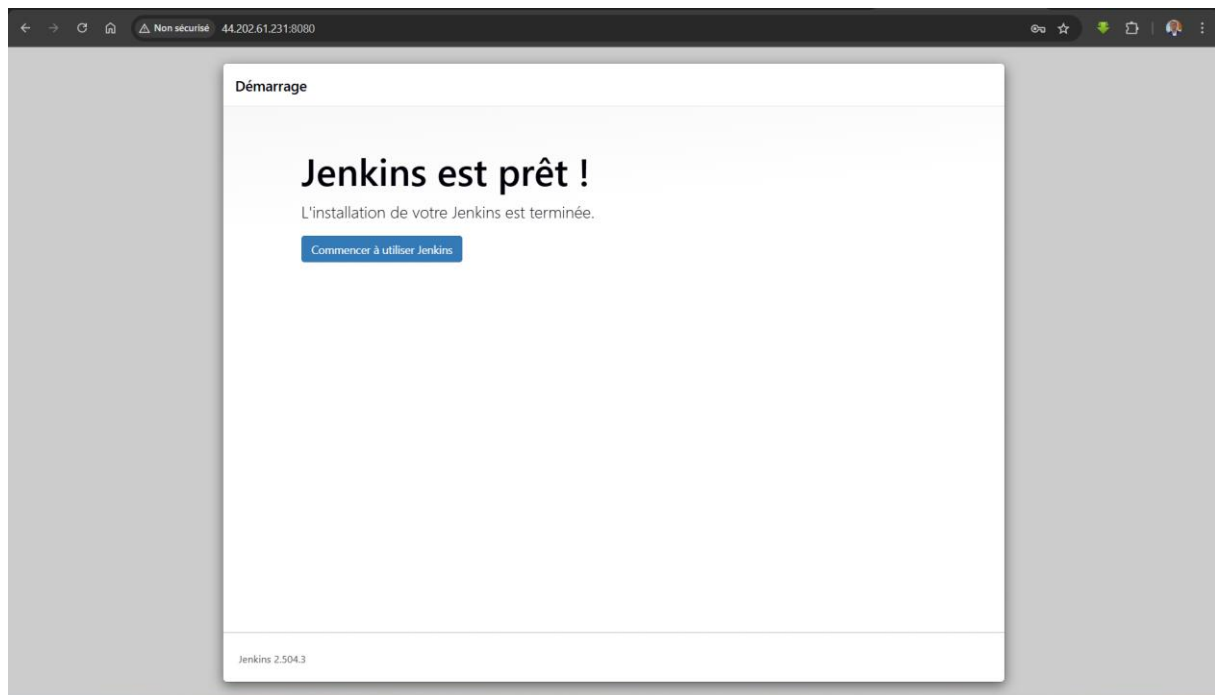
admin@net.com

Jenkins 2.504.3

Continuer en tant qu'Administrateur

Sauver et continuer

Choose default configuration of url and continue to get the next page



Finally, this is the dashboard

Installation of tools

In this step, we need to install some important tools for our exercise such as Maven and JDK. Follow the next steps :

Click to manage Jenkins

Click to tools

Scroll down and click to add JDK

Give the name JDK17

Uncheck install automatically if selected by default

To have Java_Home, connect to the server with ssh command

Switch to admin user : `sudo -i`

Update and install JDK17 : `apt update && apt install openjdk-17-jdk -y`

Execute the command to have jdk home : `ls /usr/lib/jvm/`

You will see two lines : `java-17-openjdk-amd64` and `java-21-openjdk-amd64`

The full path of JDK17 is : `/usr/lib/jvm/java-17-openjdk-amd64`

Give a name and choose the version 3.9.x of Maven in the select list

Click to save to finish

Second step : launch the nexus server

On crée d'abord une instance d'EC2 avec les paramètres suivants :

Name : nexus-server

OS : Amazon Linux

Amazon Machine Image : Amazon Linux 2023 Kernel-6.1 AMI

Instance type : t2.medium (on a besoin de plus de ressources)

Key pair : nexus-key.pem

Security group : name(nexus-sg), description(nexus-sg)

Allow ssh inbound rule from MyIP

Allow Custom TCP with port 8081 from MyIP

Allow Custom TCP with port 8081 from Jenkins-SG

In the github repo, branch Jenkins_CI, copy the content of the file userdata/nexus-setup.sh and paste it to the User data file of the Advance details section to setup-up the server while launching the instance

Launch the instance to get the server

To verify if everything is Ok, we will check nexus status, java -version et and the conf file in /opt/nexus/. So, we need to :

Connect to the instance with ssh

Switch to root user : sudo -i

Check nexus status : systemctl status nexus the status is running

ls /opt/nexus/

java -version

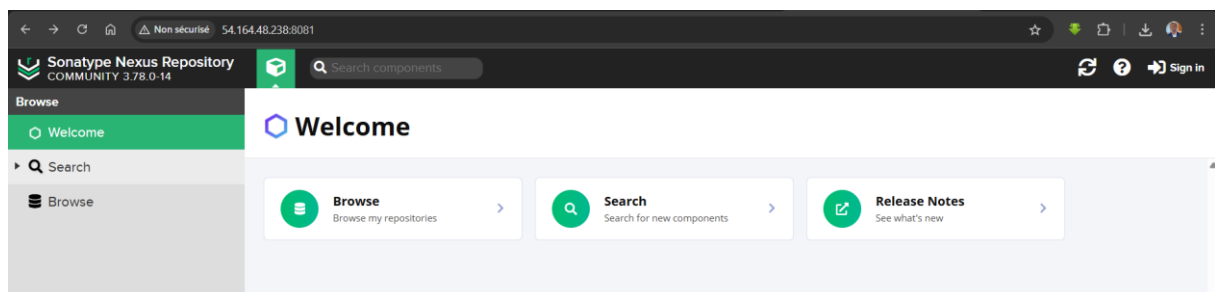
```
root@ip-172-31-91-190:~
elidj@Sougoumay MINGW64 ~/OneDrive/Bureau/DevOps/aws/project/vprofile-project (jenkins_CI)
$ ssh -i "nexus-key.pem" ec2-user@ec2-54-164-48-238.compute-1.amazonaws.com
The authenticity of host 'ec2-54-164-48-238.compute-1.amazonaws.com (54.164.48.238)' can't be established.
ED25519 key fingerprint is SHA256:7Qjm+/zfQBfJlgwNkKZtNlmbIGUMZILE4E1bX0DjSqY.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:55: 54.164.48.238
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-54-164-48-238.compute-1.amazonaws.com' (ED25519) to the list of known hosts.

#####
#                                     Amazon Linux 2023
#                                     https://aws.amazon.com/linux/amazon-linux-2023
#
#####

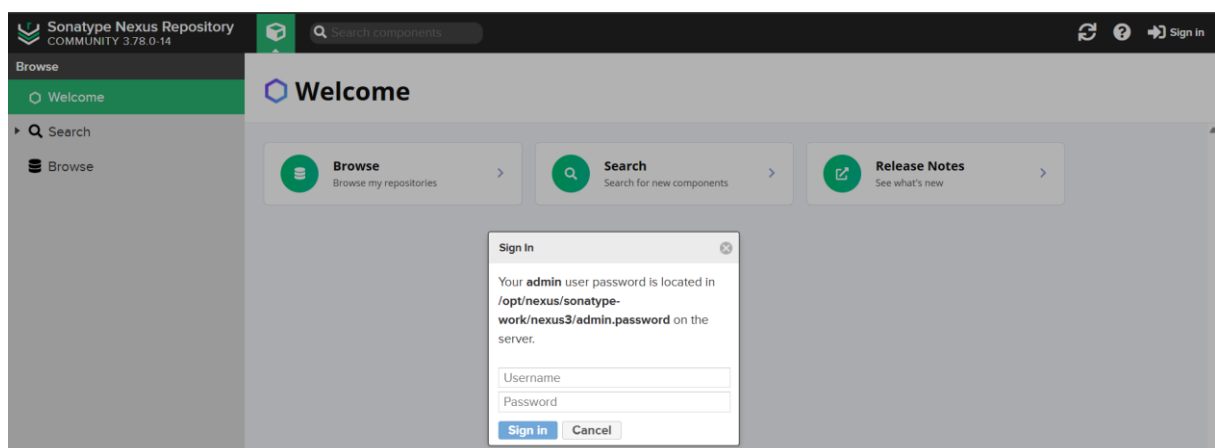
[ec2-user@ip-172-31-91-190 ~]$ sudo -i
[root@ip-172-31-91-190 ~]# systemctl status nexus
● nexus.service - nexus service
   Loaded: loaded (/etc/systemd/system/nexus.service; enabled; preset: disabled)
   Active: active (running) since Sun 2025-06-29 10:09:16 UTC; 11min ago
     Main PID: 26256 (java)
       Tasks: 73 (limit: 4656)
      Memory: 2.0G
         CPU: 1min 27.474s
        CGroup: /system.slice/nexus.service
                └─26256 /opt/nexus/nexus-3.78.0-14/jdk/temurin_17.0.13_11_linux_x86_64/jdk-17.0.13+11/bin/java -server

Jun 29 10:09:16 ip-172-31-91-190.ec2.internal systemd[1]: Starting nexus.service - nexus service...
Jun 29 10:09:16 ip-172-31-91-190.ec2.internal nexus[26028]: Starting nexus
Jun 29 10:09:16 ip-172-31-91-190.ec2.internal systemd[1]: Started nexus.service - nexus service.
[root@ip-172-31-91-190 ~]# ls /opt/nexus/
nexus-3.78.0-14  sonatype-work
[root@ip-172-31-91-190 ~]# java -version
openjdk version "17.0.15" 2025-04-15 LTS
OpenJDK Runtime Environment Corretto-17.0.15.6.1 (build 17.0.15+6-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.15.6.1 (build 17.0.15+6-LTS, mixed mode, sharing)
[root@ip-172-31-91-190 ~]# |
```

Now we need to check the connexion to the front through the browser in port 8081



Click to the sign in button to have instructions to get username and password

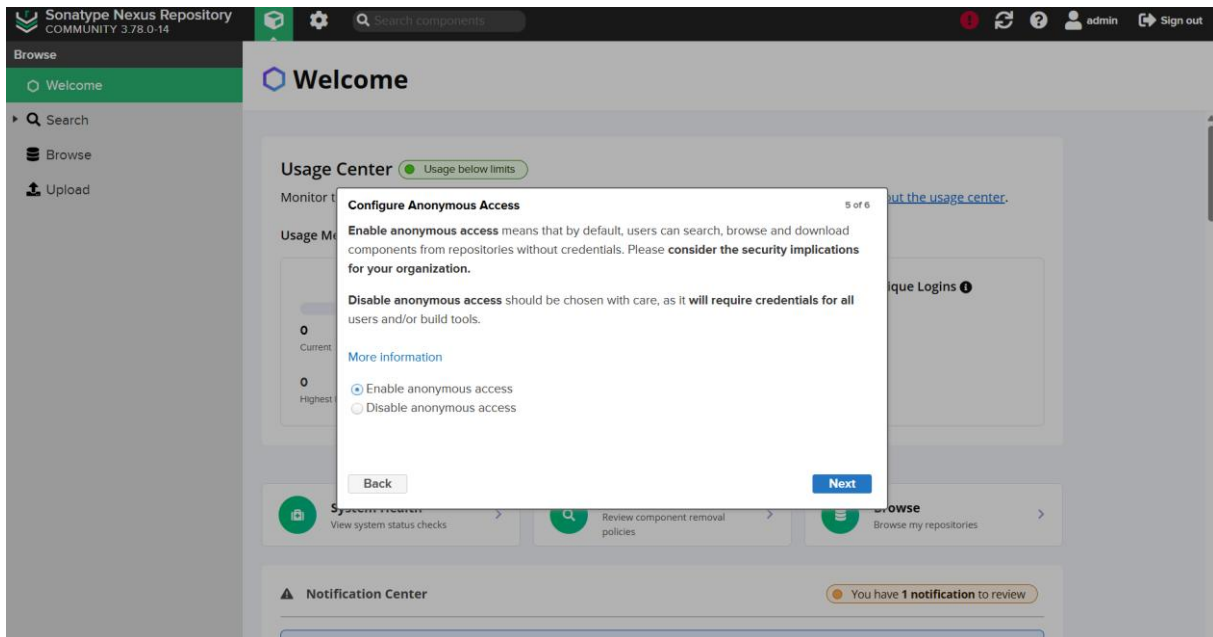


In our case, the command to get the password for admin user is : `cat /opt/nexus/sonatype-work/nexus3/admin.password`

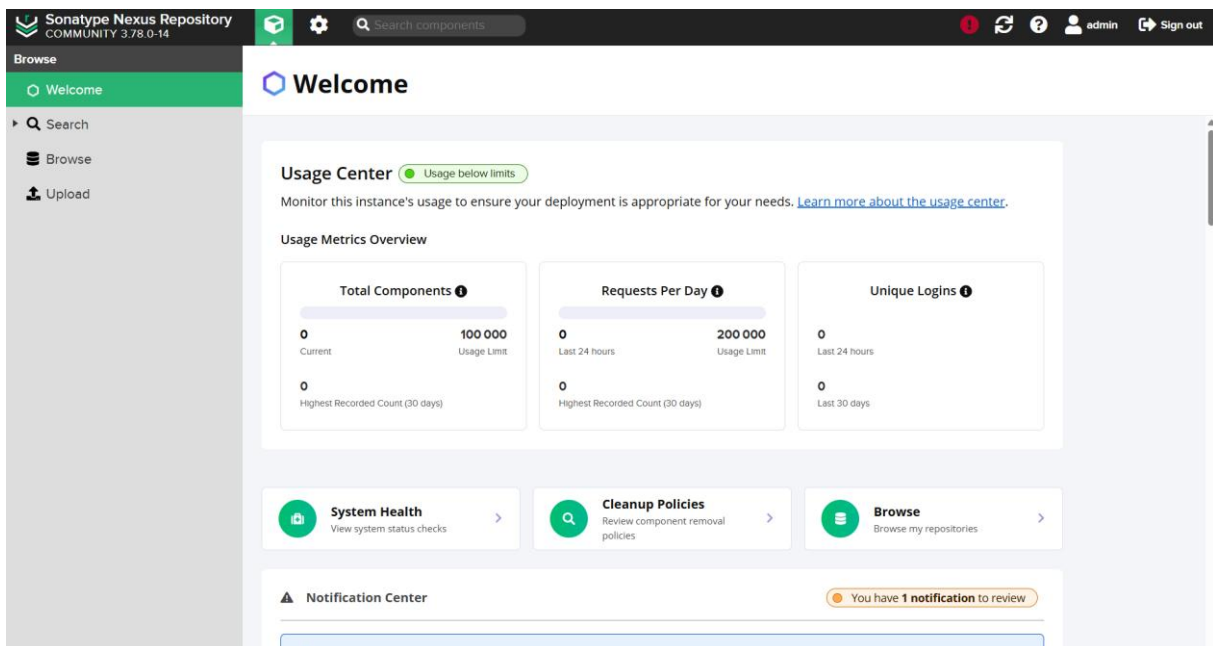
Copy your password and connect as admin

Change your password to admin123 (this is password is used in the next step within the pipeline)

Enable anonymous access



Here we go, that is the admin dashboard



Third step : launch the sonar server

Firstly, we need to launch an EC2 instances with the next parameters :

Name : sonar-server

OS : Ubuntu

Amazon Machine Image : Ubuntu 24.04

Instance type : t2.medium (we need more resources than the micro type)

Key pair : sonar-key.pem

Security group : name(sonar-sg), description(sonar-sg)

Allow ssh inbound rule from MyIP

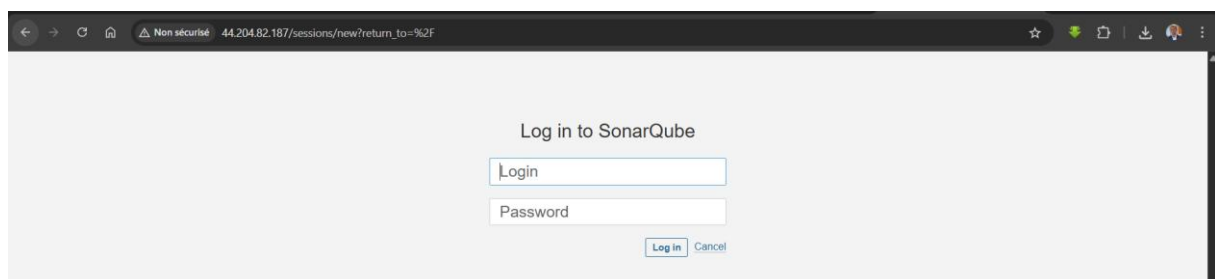
Allow Custom TCP with port 80 from MyIP (because we have nginx in the front)

Allow Custom TCP with port 80 from jenkins-sg with description Allow Jenkins to communicate

In the github repo, branch Jenkins_CI, copy the content of the file userdata/sonar-setup.sh and paste it to User data file of Advance details section to setup-up the server while launching the instance

Launch the instance to get the server

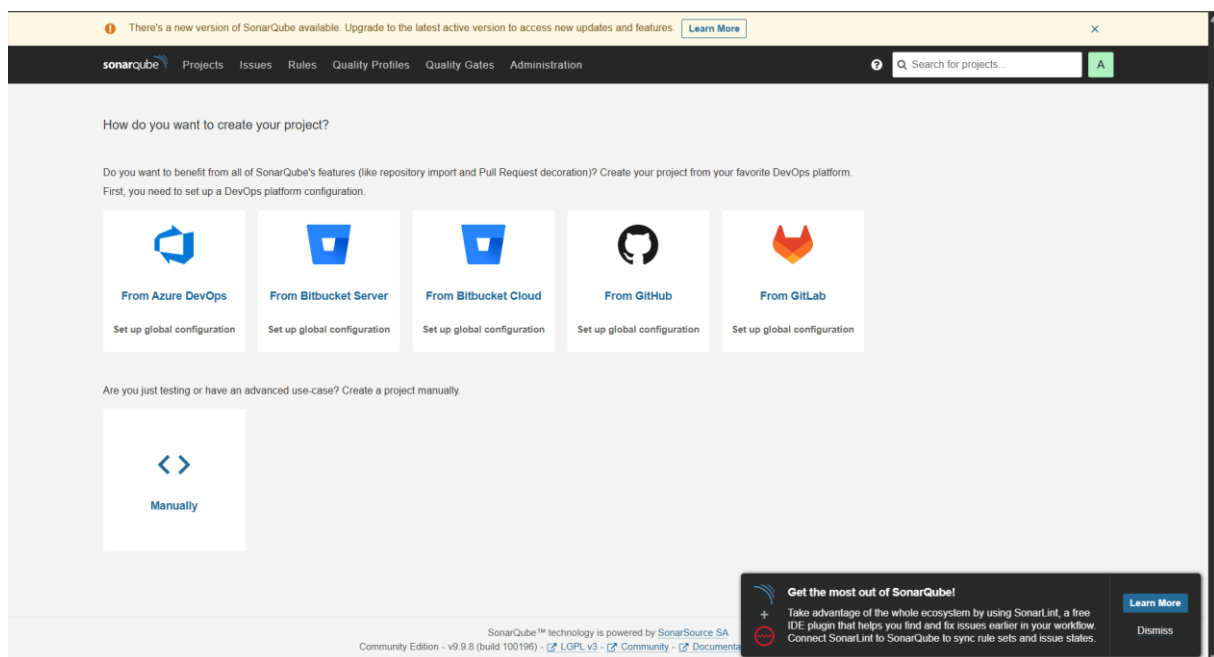
Now check if sonar is running by using the browser (take the public IP address and paste in the browser, no need to specify port because the nginx front is running in port 80)



The login is admin and the password is admin

The next step is to change the password, I chose admin123

Now we are connected, this is the dashboard



Install Plugins for CI

In this step, we gonna install plugins for Sonar, Nexus, BuildTimestamp, Pipeline Maven Integration.

To do that, connect into the Jenkins Server, go to Manage Jenkins, click into Manage Plugins, switch to available, search sonarqube scanner, nexus artifact uploader, build timestamp, pipeline maven integration, pipeline utility steps and check them all. Then click to install

The next is to execute a pipeline with the next code

```

pipeline {
    agent any

    tools {
        maven "MAVEN3.9"
        jdk "JDK17"
    }

    stages {
        stage('Fetch code') {
            steps {
                git branch: 'jenkins_CI',
                url: 'https://github.com/Sougoumay/vprofile-project'
            }
        }

        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn install -DskipTests'
            }
            post {
                success {
                    echo 'Archiving artifact'
                    archiveArtifacts artifacts: '**/*.war'
                }
            }
        }
    }
}

```

Go to the dashboard (Jenkins in web)

New Item

Give a name

Select pipeline

Click OK

Paste the code on the script input

Execute

Save

Launch the build

Code Analysis

SonarQube Scanner tool

Sonar server details

From sonar we need token

To analyze the code with SonarQube, Jenkins and SonarQube must be in run. The we need three things :

SonarQube Scanner tool in Jenkins

Sonar server details in Jenkins

Token from Sonarqube to authorize Jenkins taking actions

First of all, to install Sonarqube Scanner, go to Jenkins dashboard

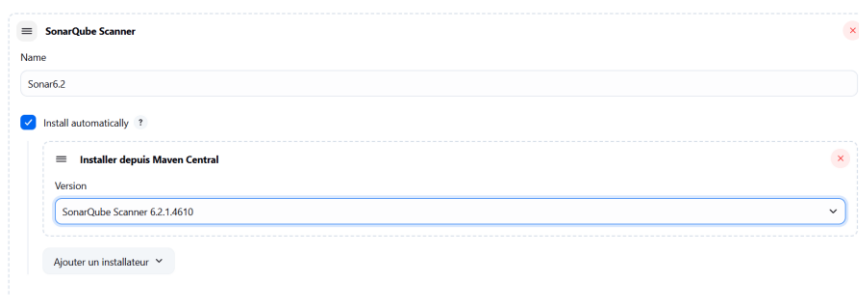
Click to manage Jenkins

Click to tools

Scroll down until SonarQube scanner and click to Add SonarQube Scanner

Give the name Sonar6.2

Choose the version 6.2.1.4610



The screenshot shows the 'SonarQube Scanner' configuration window in Jenkins. It includes a 'Name' field with the value 'Sonar6.2', a checked 'Install automatically' checkbox, and an 'Installer depuis Maven Central' section. Within this section, the 'Version' dropdown menu is open, showing 'SonarQube Scanner 6.2.1.4610' as the selected option. At the bottom, there is a button labeled 'Ajouter un installateur'.

In the next step, we need to store Sonarqube Server details in the Jenkins. SonarQube Server tool will scan the code and then we need to upload result to sonarqube server. For that we need to store the information of sonarqube server. Follow the next steps

Go to manage Jenkins

Go to System

Scroll down until you find Sonarqube server

Check environment variables

Click on Add Sonarqube

Give the name sonarserver

Server url will be the private ip of sonar, go to AWS, copy and paste (<http://private-ip>)

For the authentication we need Sonar token

Now connect to sonar server with public ip in the browser

Login with your credentials (admin and admin123)

In the top right click to the symbol A

Click to my account

Click to security

Give the name Jenkins

Select User Token

Then Generate

There's a new version of SonarQube available. Upgrade to the latest active version to access new updates and features. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration ? Search for projects... A

A Administrator Profile Security Notifications Projects

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name: Type: Expires in:

Name	Type	Project	Last use	Created	Expiration
No tokens					

New token "jenkins" has been created. Make sure you copy it now, you won't be able to see it again!

Copy

squ_bb14910c998b6209042353b8d62acd490ab4c55e

Name	Type	Project	Last use	Created	Expiration	
jenkins	User		Never	July 4, 2025	August 3, 2025	Revoke

Copy your token (if you olost, you have to remove this one and generate a new one)

Go to Jenkins

Click to add (in the authentication section of sonarqube servers(Jenkins))

Select the type secret text

Paste the token in the secret field

Give sonartoken as ID and description

Click to add

Then choose sonarqube in the list

Installations de SonarQube
Liste des installations de SonarQube

Nom

sonarserver

URL du serveur

Par défaut à http://localhost:9000

http://172.31.95.76

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonartoken

+ Ajouter

Avancé

Ajouter une installation SonarQube

Be sure to authorize Jenkins to communicate with sonar by adding an inbound rule on sonar security group, with type http

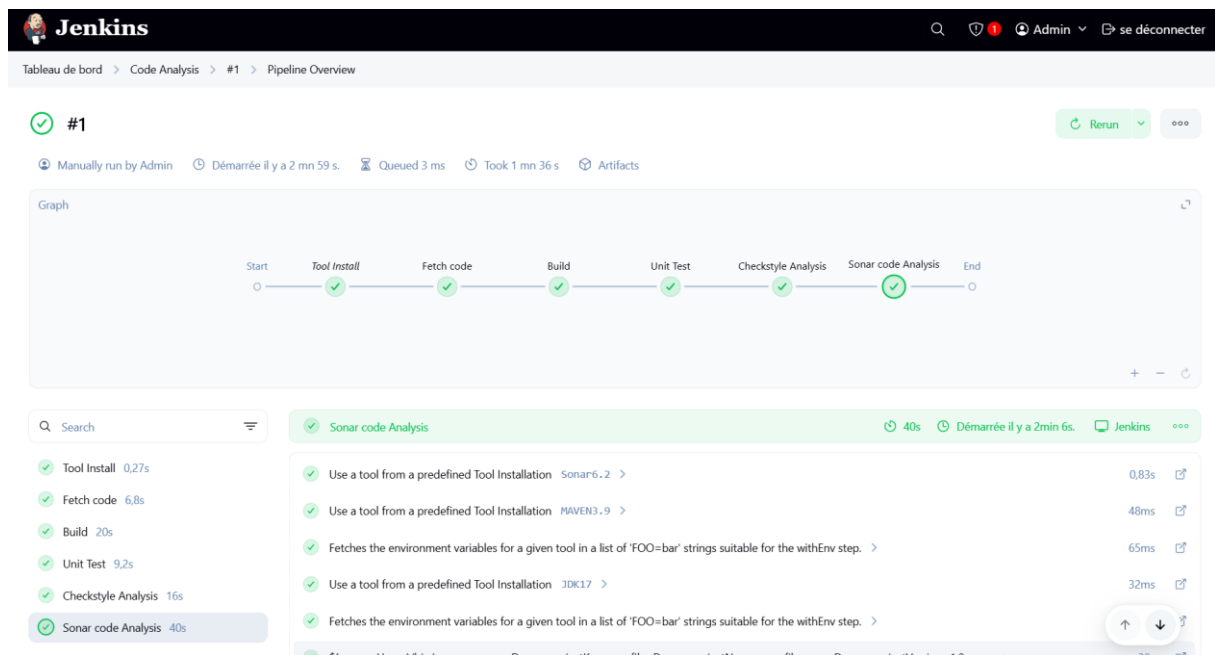
In the code of the pipeline we executed before, put the build stage before the unit test stage, then add this new stage

```
stage('Checkstyle Analysis') {
    steps {
        sh 'mvn checkstyle:checkstyle'
    }
}

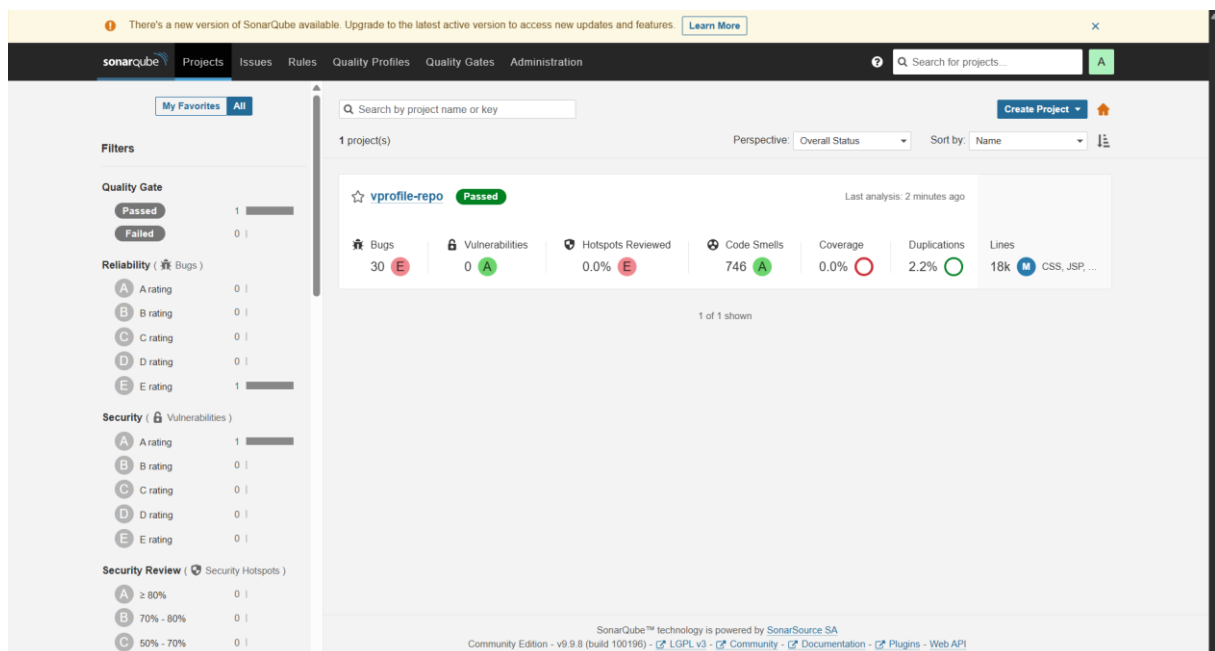
stage('Sonar code Analysis') {
    environment {
        scannerHome = tool 'Sonar6.2'
    }
    steps {
        withSonarQubeEnv('sonarserver') {
            sh "${scannerHome}/bin/sonar-scanner -Dsonar.projectKey=vprofile \
            -Dsonar.projectName=vprofile-repo \
            -Dsonar.projectVersion=1.0 \
            -Dsonar.sources=src/ \
            -Dsonar.java.binaries=target/test-classes/com/visualpathit/account/controllerTest/
            \
            -Dsonar.junit.reportsPath=target/surefire-reports/ \
            -Dsonar.jacoco.reportsPath=target/jacoco.exec \
            -Dsonar.java.checkstyle.reportPaths=target/checkstyle-result.xml"
        }
    }
}
```

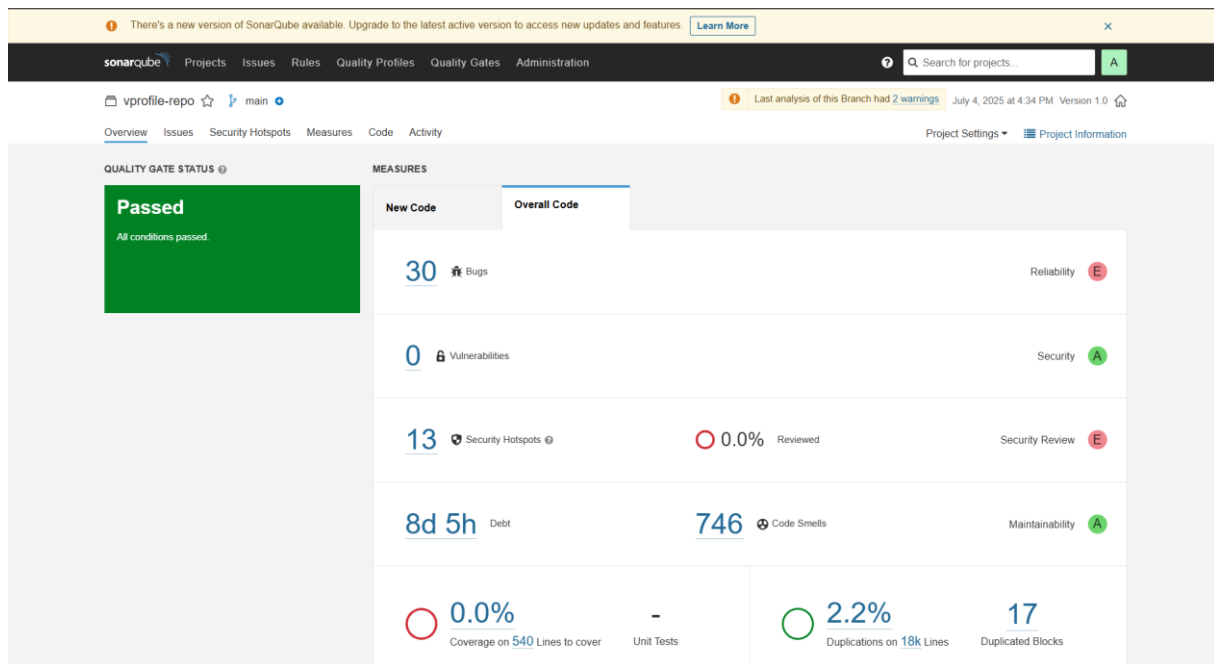
Now, create a new item (pipeline), copy paste this code and run the pipeline

Here we go, our pipeline is executed successfully



Go to sonar server and you will have this picture





Everything is good and the pipeline passed, this is the default quality gate. In the next step, we will define Quality Gate to determine when the analysis will be a success or not.

To define a quality gate,

Go to SonarQube dashboard

Go to quality gate section

Click to create button

Give a name : Jenkins-QG

Save to create the quality gate

Scroll Down

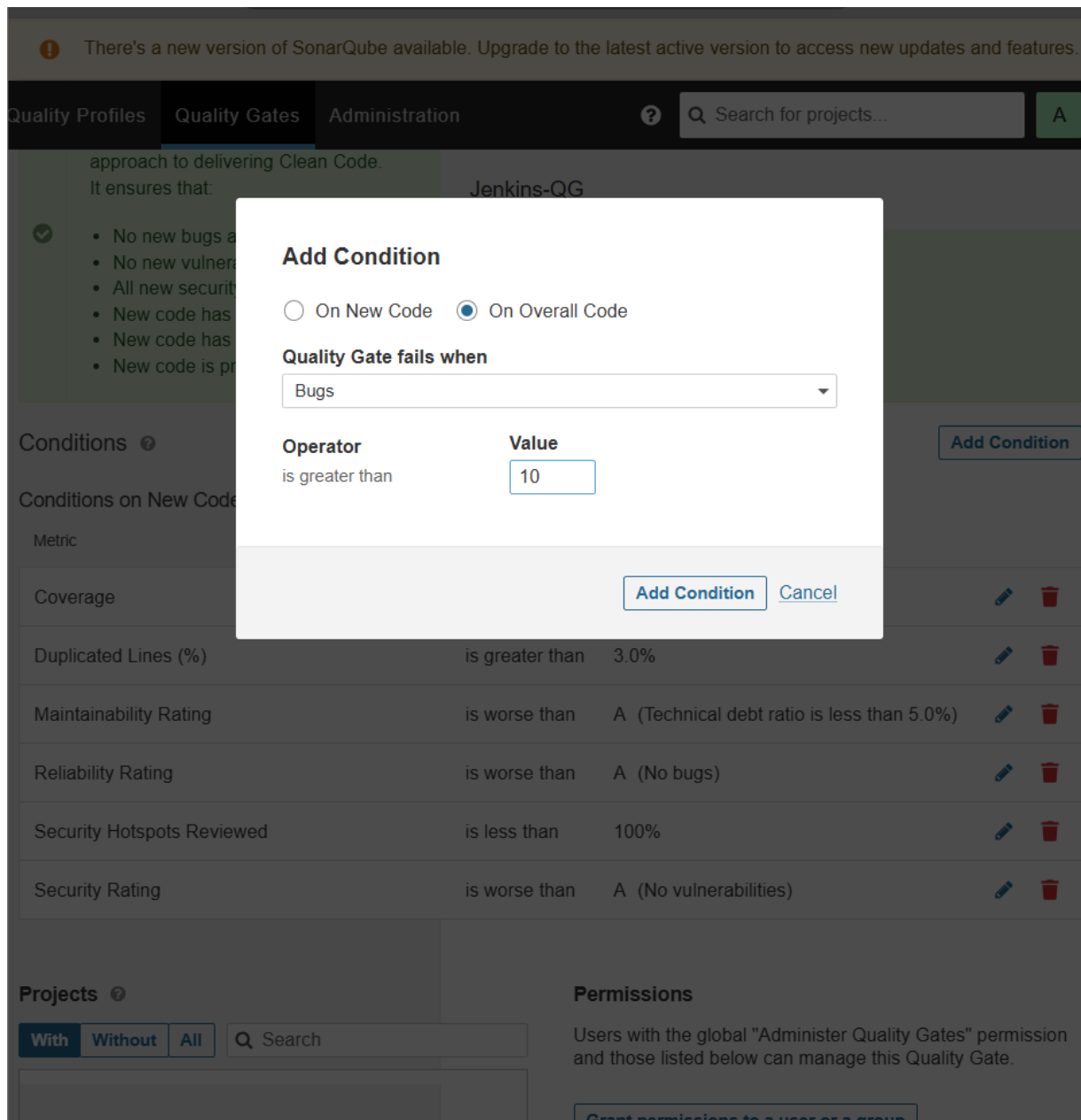
Click to Unlock editing

Click to Add Condition

Choose On Overall Code

In Quality Gate fails when, you have lots off condition, in this exercise, I choosed Bugs

Give 10 as Value (There are 30 bugs, so this time, the result will not pass)



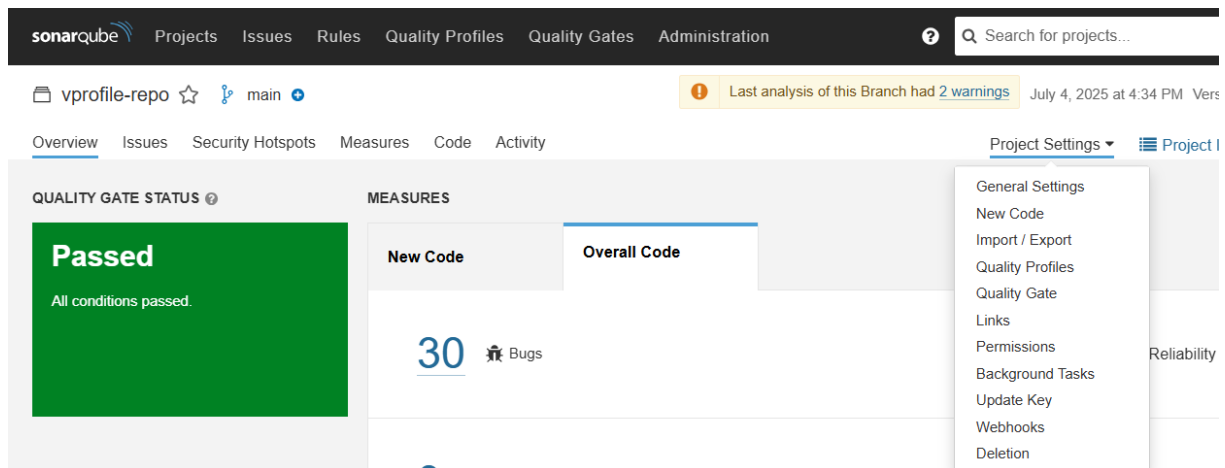
After that

Go to Projects Section

Choose our project (vprofile-repo)

Click to project settings

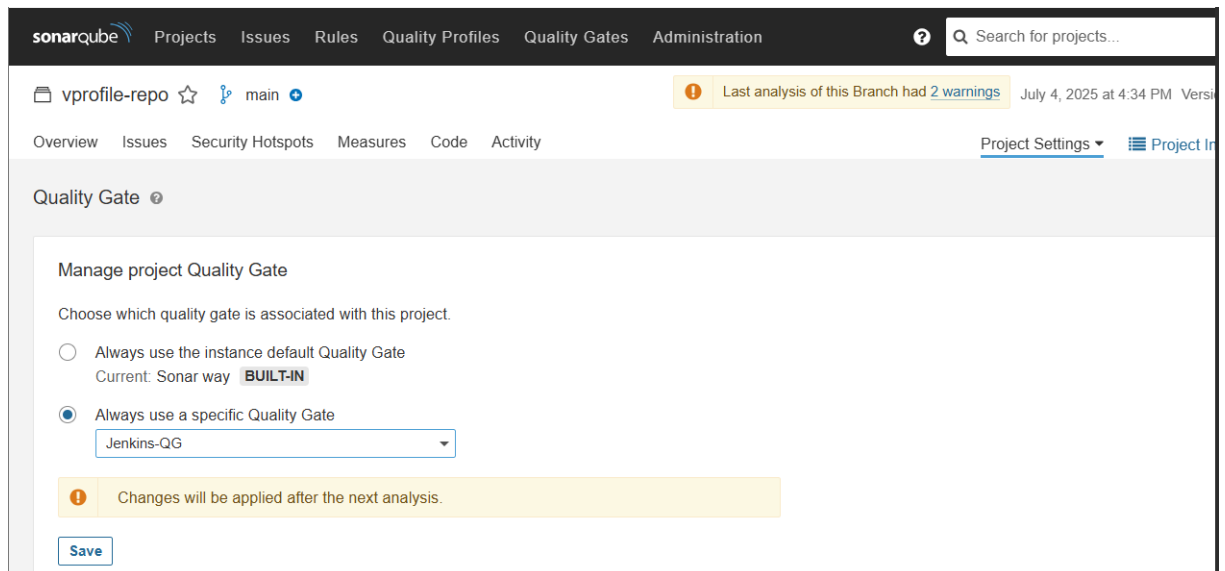
Click to quality gate



Choose Always use a specific quality gate

Select Jenkins-QG

Then Save



Once the quality gate is checked, sonarqube will sent the result to Jenkins. So we need to create a webhook. Still on Sonarqube,

choose the project,

click to project settings

lick to webhook to create one

Click to create

Give the name : Jenkins-CI-WebHook

The url will be <http://private-ip-of-jenkins:8080/sonarqube-webhook>

Click on create

Create Webhook

All fields marked with * are required

Name *



URL *



Server endpoint that will receive the webhook payload, for example: "http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: "https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create

[Cancel](#)

Add this stage on the pipeline

```
stage("Quality Gate") {
    steps {
        timeout(time: 1, unit: 'HOURS') {
            waitForQualityGate abortPipeline: true
        }
    }
}
```

```
}  
  
}
```

Update our pipeline code on Jenkins to adding this stage below, to do that

Go to the dashboard of Jenkins

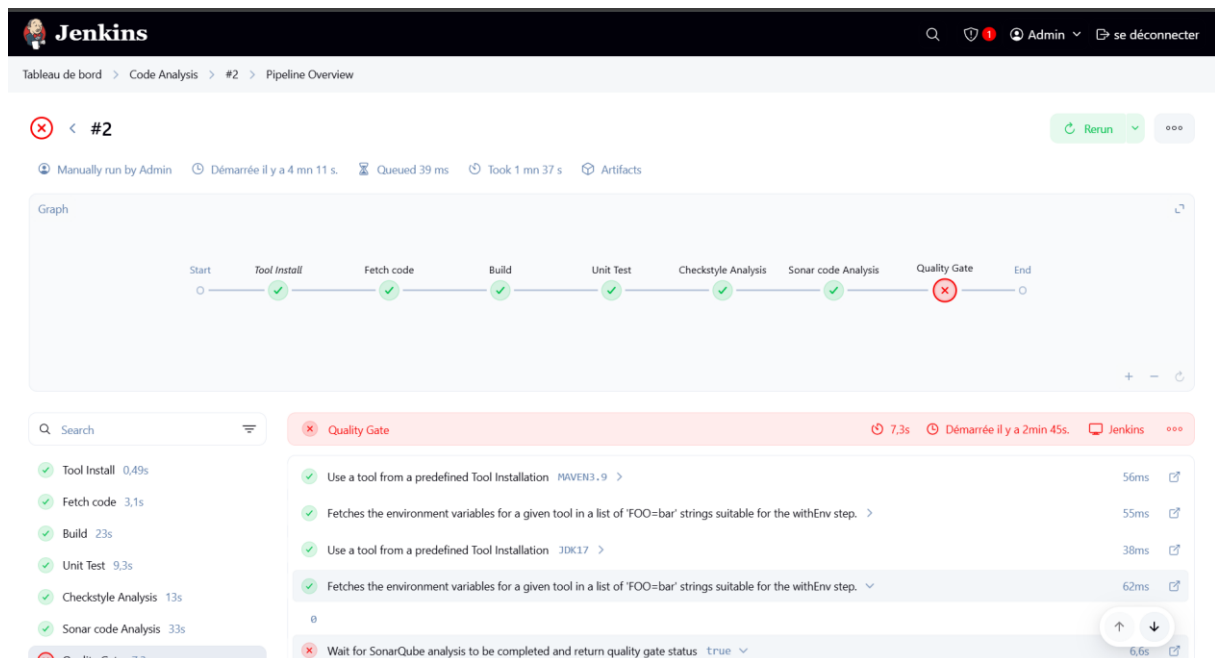
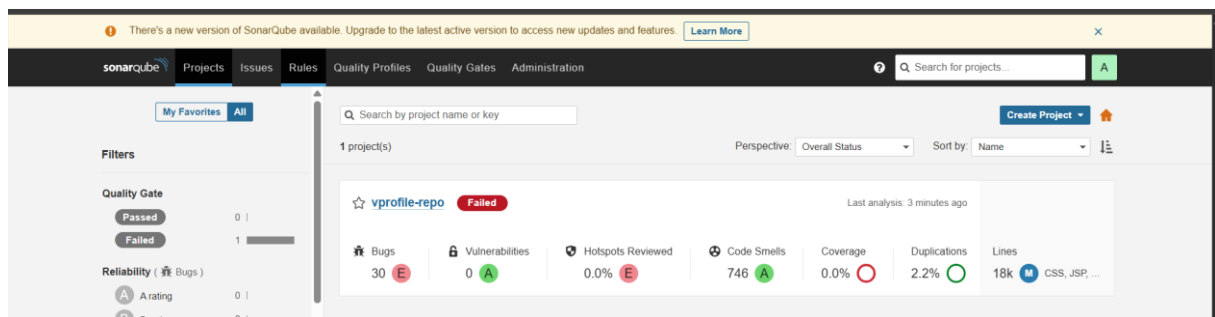
Click to the Analysis Code Pipeline

Click to configure

Scroll down, put the code at the end of the Sonar code analysis stage

Save and Run the build

Now, the pipeline will fail because there are more than 10 Bugs, SonarQube Analysis will not pass, so the last stage of the pipeline will fail

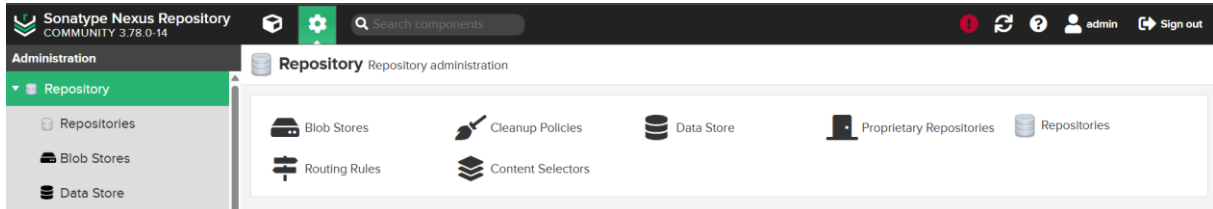


In the next step, we will store the artefacts in Nexus Reposiotry. To have a success pipeline, I switch the quality gate to the default.

Store Artifacts to Nexus Repository

First of all, run the Sonar server, take the public ip and connect to the browser with port 8081

Connect as admin (username = admin, password = admin123)



Click to the settings icon

Click to Repository

Click to create repository, scroll down and choose maven2(hosted)

Give the name : vprofile-repo and create the repository

Now, to allow Jenkins to push the artifacts to Nexus, we need to add Sonar credentials to Jenkins, follow the next steps

Go to Jenkins dashboard

Click to manage Jenkins

Scroll down and click to Credentials

In the stores scoped to Jenkins, click to System link

Click to global credentials

Click to Add credentials

Set (username = admin, password = admin123, ID = Description = nexuslogin)

Click to create

Tableau de bord > Administrer Jenkins > Identifiants > System > Identifiants globaux (illimité) >

New credentials

Type

Nom d'utilisateur et mot de passe

Portée ?

Global (Jenkins, agents, items, etc...)

Nom d'utilisateur ?

admin

☐ Treat username as secret ?

Mot de passe ?

.....

ID ?

nexuslogin

Description ?

nexuslogin

Create

The next step is to set up the stage to upload the artifacts to Nexus

Click to this link to show an exemple of the above code (Jenkins pipeline example)

Add this stage at the pipeline

```
stage('Upload Artifact') {
  steps {
    nexusArtifactUploader(
      nexusVersion: 'nexus3',
      protocol: 'http',
      nexusUrl: '172.31.91.190:8081',
      groupId: 'QA',
      version: '${env.BUILD_ID}-${env.BUILD_TIMESTAMP}',
      repository: 'vprofile-repo',
      credentialsId: 'nexuslogin',
      artifacts: [
        [artifactId: 'vproapp',
         classifier: '',
         file: 'target/vprofile-v2.war',
         type: 'war'
```



```

    ]
  ]
)
}
}

```

Go to Jenkins and create a new pipeline

Some times you will have this error, that means there is no node to execute the task

 **Sortie de la console**

 Download
  Copy
 Afficher ous forme de texte simple

```

Started by user Admin
[Pipeline] Start of Pipeline
[Pipeline] node
Still waiting to schedule task
Waiting for next available executor
Aborted by Admin
[Pipeline] // node
[Pipeline] End of Pipeline

```

To fix it,

connect to Jenkins server using ssh

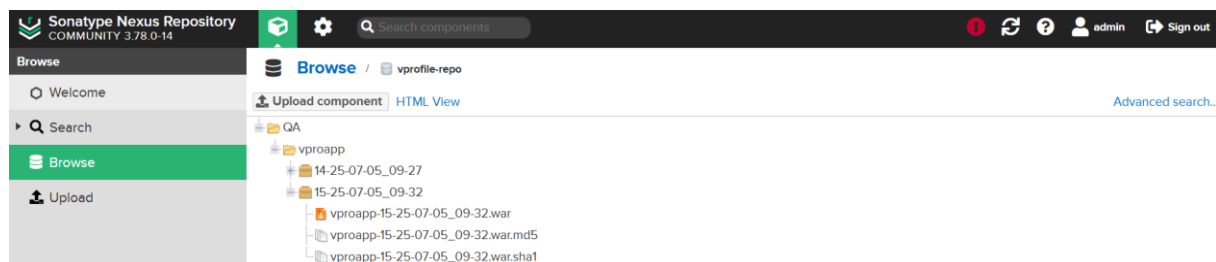
switch to rrot user : sudo -i

Go to Workspace o Jenkins path : cd /var/lib/Jenkins/workspace

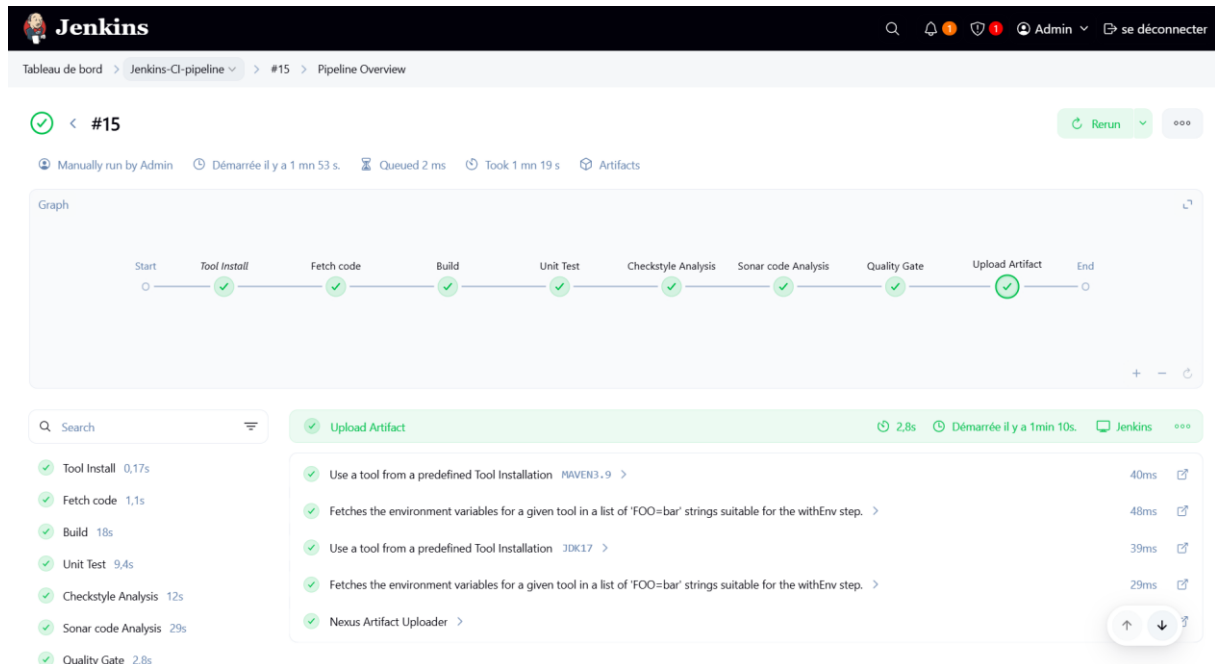
Delete all pipeline ; rm -rf * (Be sur to be under the folder workspace

Restart Jenkins : systemctl restart jenkins

Finally, the pipeline is a success and the artifacts are uploaded into Nexus



The screenshot shows the Sonatype Nexus Repository interface. The top navigation bar includes the Sonatype logo, version information (COMMUNITY 3.78.0-14), a search bar, and user controls (admin, Sign out). The left sidebar has a 'Browse' menu with options: Welcome, Search, Browse (selected), and Upload. The main content area shows the 'Browse' view for the 'vprofile-repo' repository. It displays a tree structure with a 'QA' folder containing a 'vproapp' folder. Inside 'vproapp', there are two subfolders: '14-25-07-05_09-27' and '15-25-07-05_09-32'. The '15-25-07-05_09-32' folder contains three files: 'vproapp-15-25-07-05_09-32.war', 'vproapp-15-25-07-05_09-32.war.md5', and 'vproapp-15-25-07-05_09-32.war.sha1'.



Notification with Slack

First of all, you need to have a slack account connected to the browser.

To configure Jenkins to send notification to slack channel, go to the market place of slack (<https://your-slack-account.slack.com/marketplace>)

Search Jenkins

Click to Jenkins CI

Click to Add Slack

Choose a channel in the select list or create one

Click to Add Jenkins CI integration

And you will have all the next steps in the next page of slack

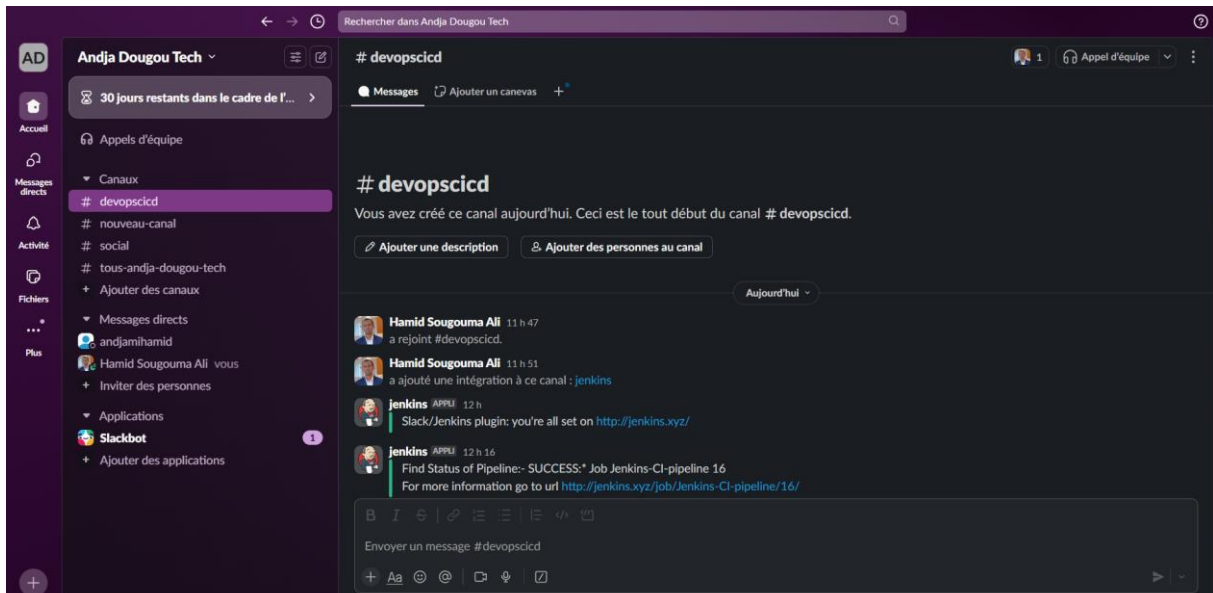
After this configuration, you will add to the pipeline a post section after stages section to send the notification.

```
post {
    always {
        echo 'Slack Notification'
        slackSend channel: '#devopscicd',
            color: COLOR_MAP[currentBuild.currentResult],
            message: "Find Status of Pipeline:- ${currentBuild.currentResult}:* Job  
${env.JOB_NAME} ${env.BUILD_NUMBER} \n For more information go to url  
${BUILD_URL}"
    }
}
```

The function below is to put before pipelines section

```
def COLOR_MAP = [
    'SUCCESS': 'good',
    'FAILURE': 'danger'
]
```

The pipeline is a success and I received the notification in slack



Graph

```
graph LR; Start((Start)) --> ToolInstall[Tool Install]; ToolInstall --> FetchCode[Fetch code]; FetchCode --> Build[Build]; Build --> UnitTest[Unit Test]; UnitTest --> Checkstyle[Checkstyle Analysis]; Checkstyle --> Sonar[Sonar code Analysis]; Sonar --> QualityGate[Quality Gate]; QualityGate --> UploadArtifact[Upload Artifact]; UploadArtifact --> PostActions[Post Actions]; PostActions --> End((End));
```

Post Actions 0,56s Démarrée il y a 53s. Jenkins

- | Step | Operation | Duration |
|------|--------------------|----------|
| 1 | Slack Notification | 38ms |
| 2 | Send Slack Message | 0.47s |